FinAssist AI

Assistente Virtual Inteligente para Relacionamento Financeiro

Projeto desenvolvido em Python com Flask, aplicando IA, dados e UX

1. Visão Geral do Projeto

O **FinAssist AI** é um assistente virtual desenvolvido para atender ao desafio de criação de uma experiência digital voltada ao relacionamento financeiro, guiada por inteligência artificial e fundamentada em boas práticas de experiência do usuário.

A solução integra compreensão de linguagem natural, respostas contextualizadas, simulações financeiras simples e persistência básica de contexto, consolidando aprendizado em IA, Python, dados e UX.

2. Objetivos da Solução

- Compreender perguntas em linguagem natural
- Responder dúvidas frequentes sobre produtos financeiros
- Realizar simulações demonstrativas de empréstimos e investimentos
- Manter contexto básico durante a conversa
- Aplicar princípios de clareza, segurança e transparência na experiência do usuário

3. Funcionalidades Implementadas

- FAQ inteligente com base em palavras-chave
- Simulação de empréstimo com parcelas estimadas (juros compostos)
- Simulação de investimento com capitalização mensal
- Continuidade de contexto em mensagens subsequentes
- Interface web amigável e responsiva
- Reinício de conversa

4. Arquitetura da Solução

A aplicação foi estruturada de forma modular para facilitar manutenção, evolução e clareza do código.

Componentes principais

Componente	Responsabilidade
app.py	Inicializa a aplicação Flask e expõe as rotas
intent_classifier.py	Identifica a intenção da mensagem do usuário
financial.py	Executa cálculos financeiros (empréstimo e investimento)
context_manager.py	Armazena e gerencia contexto da conversa
response_generator.py	Gera respostas com base na intenção e no contexto

faq_data.json	Base de conhecimento com perguntas frequentes
templates/index.html	Interface principal do chat
static/style.css	Estilização da interface

Fluxo da aplicação

1. O usuário envia uma mensagem pela interface web
2. A aplicação identifica a intenção da mensagem
3. O sistema consulta a base de FAQs ou executa uma simulação financeira
4. O contexto da conversa é atualizado
5. A resposta é retornada ao usuário de forma clara e objetiva

5. Tecnologias Utilizadas

- Python 3
- Flask (framework web)
- HTML5
- CSS3
- JavaScript
- JSON (base de conhecimento)

6. Estrutura do Projeto

```
finassist-ai/
├── app.py
├── financial.py
├── intent_classifier.py
├── context_manager.py
├── response_generator.py
├── faq_data.json
├── requirements.txt
├── README.md
├── .gitignore
├── templates/
│   └── index.html
└── static/
    └── style.css
```

7. Fórmulas Financeiras Utilizadas

Empréstimo com parcelas fixas (Sistema Price)

Fórmula:

```
PMT = PV * [i(1+i)^n] / [(1+i)^n - 1]

Onde:
- PV = valor principal (valor do empréstimo)
- i = taxa de juros mensal (em decimal)
- n = número de parcelas
```

Investimento com juros compostos

Fórmula:

```
M = C(1+i)^n

Onde:
- C = capital inicial
- i = taxa mensal (em decimal)
- n = número de meses
- M = montante final
```

8. Exemplos de Uso

FAQ

- O que é renda fixa?
- Como funciona cartão de crédito?
- O que são juros?

Simulação de empréstimo

- Quero um empréstimo de 5000 com juros de 2% por 12 meses

Simulação de investimento

- Investir 1000 a 1% ao mês por 12 meses

Continuidade de contexto

1. Investir 1000 a 1% ao mês por 12 meses
2. E se eu deixar por 24 meses?

9. UX e Segurança

O projeto foi pensado com foco em experiência do usuário e uso responsável em contexto financeiro.

Princípios aplicados

- Linguagem simples e acessível
- Clareza nas respostas
- Sinalização de que as simulações são demonstrativas

- Não solicitação de dados sensíveis (senha, token, etc.)
- Transparência sobre limitações da solução
- Respeito aos princípios de design conversacional

Disclaimers utilizados

Exemplo de disclaimer em simulação de empréstimo:
"Importante: esta simulação é apenas demonstrativa e pode variar conforme análise de crédito, perfil do cliente e condições contratuais."

Exemplo de disclaimer em simulação de investimento:
"Importante: este cálculo é ilustrativo e não representa garantia de rentabilidade futura."

10. Diferenciais do Projeto

- Arquitetura modular e organizada
- Separação clara de responsabilidades
- Interface web amigável e responsiva
- Contexto básico de conversa implementado
- Aplicação prática de IA, Python, dados e UX
- Código limpo e bem documentado
- Pronto para evolução e integração com APIs reais

11. Código Completo dos Arquivos

app.py

```
from flask import Flask, render_template, request, jsonify, session
from intent_classifier import identificar_intencao
from context_manager import ContextManager
from response_generator import gerar_resposta

app = Flask(__name__)
app.secret_key = "finassist-secret-key"


def get_context():
    if "contexto" not in session:
        session["contexto"] = {
            "assunto_atual": None,
            "ultimo_valor": None,
            "ultimo_prazo": None,
            "ultima_taxa": None,
            "historico": []
        }
    return ContextManager.from_dict(session["contexto"])


def save_context(contexto):
    session["contexto"] = contexto.to_dict()


@app.route("/")
def home():
    return render_template("index.html")


@app.route("/chat", methods=["POST"])
def chat():
    data = request.get_json()
    mensagem = data.get("message", "").strip()

    if not mensagem:
        return jsonify({"reply": "Por favor, digite uma mensagem."})

    contexto = get_context()
    contexto.atualizar_historico(mensagem)

    intencao = identificar_intencao(mensagem, contexto)
    resposta = gerar_resposta(mensagem, intencao, contexto)

    save_context(contexto)

    return jsonify({"reply": resposta})


@app.route("/reset", methods=["POST"])
def reset():
    session.pop("contexto", None)
    return jsonify({"reply": "Conversa reiniciada com sucesso."})


if __name__ == "__main__":
    app.run(debug=True)
```

financial.py

```
def calcular_emprestimo(valor, taxa_mensal, meses):
    i = taxa_mensal / 100

    if meses <= 0:
        raise ValueError("O número de meses deve ser maior que zero.")

    if i == 0:
        parcela = valor / meses
    else:
        parcela = valor * (i * (1 + i) ** meses) / ((1 + i) ** meses - 1)

    total = parcela * meses
    juros = total - valor

    return {
        "parcela": round(parcela, 2),
        "total": round(total, 2),
        "juros": round(juros, 2)
    }


def calcular_investimento(valor_inicial, taxa_mensal, meses):
    i = taxa_mensal / 100

    if meses <= 0:
        raise ValueError("O número de meses deve ser maior que zero.")

    montante = valor_inicial * (1 + i) ** meses
    rendimento = montante - valor_inicial

    return {
        "montante": round(montante, 2),
        "rendimento": round(rendimento, 2)
    }
```

intent_classifier.py

```
import re


def identificar_intencao(mensagem: str, contexto=None) -> str:
    texto = mensagem.lower().strip()

    saudacoes = ["oi", "olá", "ola", "bom dia", "boa tarde", "boa noite"]
    encerramentos = ["sair", "encerrar", "tchau", "até mais", "ate mais"]
    faq_keywords = [
        "o que é", "o que eh", "como funciona", "cartão", "cartao",
        "renda fixa", "empréstimo pessoal", "emprestimo pessoal",
        "limite", "fatura", "juros", "investimento"
    ]
    emprestimo_keywords = ["empréstimo", "emprestimo", "parcela", "crédito", "credito",
"juros"]
    investimento_keywords = ["investir", "investimento", "rendimento", "aplicar",
"aplicação", "aplicacao"]

    if any(texto == s or texto.startswith(s + " ") for s in saudacoes):
        return "saudacao"

    if any(p in texto for p in encerramentos):
        return "encerramento"

    if any(p in texto for p in emprestimo_keywords) and possui_numeros(texto):
        return "simulacao_emprestimo"
```

```
    if any(p in texto for p in investimento_keywords) and possui_numeros(texto):
        return "simulacao_investimento"

    if contexto and contexto.assunto_atual == "investimento" and possui_numeros(texto):
        return "continuidade_investimento"

    if contexto and contexto.assunto_atual == "emprestimo" and possui_numeros(texto):
        return "continuidade_emprestimo"

    if any(p in texto for p in faq_keywords):
        return "faq"

    return "desconhecida"


def possui_numeros(texto: str) -> bool:
    return bool(re.search(r"\d", texto))


def extrair_numeros(texto: str):
    numeros = re.findall(r"\d+(?:[.,]\d+)?", texto)
    return [float(n.replace(",", ".")) for n in numeros]
```

context_manager.py

```
class ContextManager:
    def __init__(self, assunto_atual=None, ultimo_valor=None, ultimo_prazo=None,
ultima_taxa=None, historico=None):
        self.assunto_atual = assunto_atual
        self.ultimo_valor = ultimo_valor
        self.ultimo_prazo = ultimo_prazo
        self.ultima_taxa = ultima_taxa
        self.historico = historico if historico is not None else []

    def atualizar_historico(self, mensagem):
        self.historico.append(mensagem)

    def atualizar_contexto(self, assunto=None, valor=None, prazo=None, taxa=None):
        if assunto is not None:
            self.assunto_atual = assunto
        if valor is not None:
            self.ultimo_valor = valor
        if prazo is not None:
            self.ultimo_prazo = prazo
        if taxa is not None:
            self.ultima_taxa = taxa

    def to_dict(self):
        return {
            "assunto_atual": self.assunto_atual,
            "ultimo_valor": self.ultimo_valor,
            "ultimo_prazo": self.ultimo_prazo,
            "ultima_taxa": self.ultima_taxa,
            "historico": self.historico
        }

    @classmethod
    def from_dict(cls, data):
        return cls(
            assunto_atual=data.get("assunto_atual"),
            ultimo_valor=data.get("ultimo_valor"),
            ultimo_prazo=data.get("ultimo_prazo"),
            ultima_taxa=data.get("ultima_taxa"),
```

```
            historico=data.get("historico", [])
        )
```

response_generator.py

```
import json
from financial import calcular_emprestimo, calcular_investimento
from intent_classifier import extrair_numeros


def carregar_faq():
    with open("faq_data.json", "r", encoding="utf-8") as file:
        return json.load(file)


def responder_faq(mensagem):
    faqs = carregar_faq()
    texto = mensagem.lower()

    for item in faqs:
        for keyword in item["keywords"]:
            if keyword in texto:
                return item["response"]

    return (
        "Posso te ajudar com dúvidas sobre cartão de crédito, empréstimo pessoal, "
        "renda fixa, investimento e também fazer simulações financeiras simples."
    )


def formatar_moeda(valor):
    return f"R$ {valor:,.2f}".replace(",", "X").replace(".", ",").replace("X", ".")


def gerar_resposta(mensagem, intencao, contexto):
    try:
        if intencao == "saudacao":
            return (
                "Olá. Eu sou o FinAssist AI, seu assistente virtual financeiro.\n\n"
                "Posso te ajudar com:\n"
                "- explicação de produtos financeiros\n"
                "- simulação de empréstimo\n"
                "- simulação de investimento\n"
                "- dúvidas gerais sobre relacionamento financeiro"
            )

        if intencao == "faq":
            return responder_faq(mensagem)

        if intencao == "simulacao_emprestimo":
            numeros = extrair_numeros(mensagem)

            if len(numeros) >= 3:
                valor = numeros[0]
                taxa = numeros[1]
                meses = int(numeros[2])

                resultado = calcular_emprestimo(valor, taxa, meses)
                contexto.atualizar_contexto(assunto="emprestimo", valor=valor,
prazo=meses, taxa=taxa)

                return (
                    "Simulação demonstrativa de empréstimo:\n\n"
                    f"- Valor solicitado: {formatar_moeda(valor)}\n"
                    f"- Taxa mensal: {taxa:.2f}%\n"
                    f"- Prazo: {meses} meses\n"
                    f"- Parcela estimada: {formatar_moeda(resultado['parcela'])}\n"
                    f"- Total pago: {formatar_moeda(resultado['total'])}\n"
```

```
                    f"- Juros totais: {formatar_moeda(resultado['juros'])}\n\n"
                    "Importante: esta simulação é apenas demonstrativa e pode variar
conforme análise de crédito, perfil do cliente e condições contratuais."
                )

            return (
                "Para simular um empréstimo, informe os 3 dados principais:\n"
                "- valor desejado\n"
                "- taxa de juros mensal\n"
                "- prazo em meses\n\n"
                "Exemplo: Quero um empréstimo de 5000 com juros de 2% por 12 meses."
            )

        if intencao == "simulacao_investimento":
            numeros = extrair_numeros(mensagem)

            if len(numeros) >= 3:
                valor = numeros[0]
                taxa = numeros[1]
                meses = int(numeros[2])

                resultado = calcular_investimento(valor, taxa, meses)
                contexto.atualizar_contexto(assunto="investimento", valor=valor,
prazo=meses, taxa=taxa)

                return (
                    "Simulação demonstrativa de investimento:\n\n"
                    f"- Valor inicial: {formatar_moeda(valor)}\n"
                    f"- Rentabilidade mensal estimada: {taxa:.2f}%\n"
                    f"- Prazo: {meses} meses\n"
                    f"- Valor final estimado: {formatar_moeda(resultado['montante'])}\n"
                    f"- Rendimento estimado: {formatar_moeda(resultado['rendimento'])}
\n\n"
                    "Importante: este cálculo é ilustrativo e não representa garantia de
rentabilidade futura."
                )

            return (
                "Para simular um investimento, informe:\n"
                "- valor inicial\n"
                "- taxa mensal estimada\n"
                "- prazo em meses\n\n"
                "Exemplo: Investir 1000 a 1% ao mês por 12 meses."
            )

        if intencao == "continuidade_investimento":
            numeros = extrair_numeros(mensagem)

            valor = contexto.ultimo_valor
            taxa = contexto.ultima_taxa
            meses = contexto.ultimo_prazo

            if len(numeros) == 1:
                meses = int(numeros[0])
            elif len(numeros) >= 2:
                taxa = numeros[0]
                meses = int(numeros[1])

            if valor and taxa and meses:
                resultado = calcular_investimento(valor, taxa, meses)
                contexto.atualizar_contexto(assunto="investimento", valor=valor,
prazo=meses, taxa=taxa)

                return (
                    "Atualizei sua simulação de investimento com base no contexto
anterior:\n\n"
```

```
                    f"- Valor inicial: {formatar_moeda(valor)}\n"
                    f"- Rentabilidade mensal estimada: {taxa:.2f}%\n"
                    f"- Prazo: {meses} meses\n"
                    f"- Valor final estimado: {formatar_moeda(resultado['montante'])}\n"
                    f"- Rendimento estimado: {formatar_moeda(resultado['rendimento'])}
\n\n"
                    "Importante: esta projeção é apenas ilustrativa."
                )

        if intencao == "continuidade_emprestimo":
            numeros = extrair_numeros(mensagem)

            valor = contexto.ultimo_valor
            taxa = contexto.ultima_taxa
            meses = contexto.ultimo_prazo

            if len(numeros) == 1:
                meses = int(numeros[0])
            elif len(numeros) >= 2:
                taxa = numeros[0]
                meses = int(numeros[1])

            if valor and taxa and meses:
                resultado = calcular_emprestimo(valor, taxa, meses)
                contexto.atualizar_contexto(assunto="emprestimo", valor=valor,
prazo=meses, taxa=taxa)

                return (
                    "Atualizei sua simulação de empréstimo com base no contexto anterior:
\n\n"
                    f"- Valor solicitado: {formatar_moeda(valor)}\n"
                    f"- Taxa mensal: {taxa:.2f}%\n"
                    f"- Prazo: {meses} meses\n"
                    f"- Parcela estimada: {formatar_moeda(resultado['parcela'])}\n"
                    f"- Total pago: {formatar_moeda(resultado['total'])}\n"
                    f"- Juros totais: {formatar_moeda(resultado['juros'])}\n\n"
                    "Importante: esta simulação é apenas demonstrativa."
                )

        if intencao == "encerramento":
            return "Obrigado por usar o FinAssist AI. Se precisar de mais alguma
informação financeira, estou à disposição."

        return (
            "Não consegui entender completamente sua solicitação.\n\n"
            "Você pode, por exemplo:\n"
            "- perguntar 'o que é renda fixa?'\n"
            "- perguntar 'como funciona cartão de crédito?'\n"
            "- pedir 'simular empréstimo de 5000 com juros de 2% por 12 meses'\n"
            "- pedir 'investir 1000 a 1% ao mês por 12 meses'"
        )

    except ValueError as e:
        return f"Ocorreu um problema na simulação: {str(e)}"

    except Exception:
        return "Ocorreu um erro ao processar sua solicitação. Tente novamente com uma
mensagem mais específica."
```

faq_data.json

```
[
  {
    "keywords": ["empréstimo pessoal", "emprestimo pessoal", "o que é empréstimo", "o que
é emprestimo"],
```

```
    "response": "Empréstimo pessoal é uma linha de crédito oferecida ao cliente com
pagamento parcelado e cobrança de juros conforme contrato. Ele pode ser usado para
diferentes finalidades, mas exige atenção ao custo total da operação."
  },
  {
    "keywords": ["renda fixa", "o que é renda fixa", "investimento renda fixa"],
    "response": "Renda fixa é um tipo de investimento em que as regras de remuneração são
definidas no momento da aplicação ou seguem um índice de referência. Costuma ser mais
previsível do que investimentos de renda variável."
  },
  {
    "keywords": ["cartão de crédito", "cartao de credito", "como funciona cartão", "como
funciona cartao"],
    "response": "Cartão de crédito é um meio de pagamento que permite realizar compras
com cobrança posterior. O cliente deve acompanhar limite, vencimento da fatura e encargos
em caso de atraso ou uso do crédito rotativo."
  },
  {
    "keywords": ["juros", "o que são juros", "o que sao juros"],
    "response": "Juros representam o custo do dinheiro no tempo. Em operações de crédito,
correspondem ao valor pago pelo uso do recurso emprestado. Em investimentos, podem
representar a remuneração sobre o valor aplicado."
  },
  {
    "keywords": ["fatura", "limite", "fatura do cartão", "limite do cartão"],
    "response": "A fatura do cartão reúne os gastos realizados no período. Já o limite é
o valor máximo disponível para compras. Manter o controle desses dois pontos ajuda a
evitar endividamento e atraso no pagamento."
  }
]
```

requirements.txt

```
Flask==3.0.3
```

.gitignore

```
__pycache__/
*.pyc
.venv/
venv/
.env
.vscode/
.idea/
```

templates/index.html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>FinAssist AI</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
  <div class="container">
    <header class="header">
      <h1>FinAssist AI</h1>
      <p>Assistente Virtual para Relacionamento Financeiro</p>
    </header>

    <main class="chat-wrapper">
      <div id="chat-box" class="chat-box">
        <div class="message bot">
          <div class="bubble">
            Olá. Eu sou o <strong>FinAssist AI</strong>.<br><br>
            Posso te ajudar com:
            <ul>
              <li>explicação de produtos financeiros</li>
              <li>simulação de empréstimo</li>
              <li>simulação de investimento</li>
              <li>dúvidas gerais sobre relacionamento financeiro</li>
            </ul>
          </div>
        </div>
      </div>

      <div class="examples">
        <p><strong>Exemplos:</strong></p>
        <div class="example-list">
          <button onclick="fillExample('O que é renda fixa?')">O que é renda fixa?</
button>
          <button onclick="fillExample('Quero um empréstimo de 5000 com juros de 2% por
12 meses')">Simular empréstimo</button>
          <button onclick="fillExample('Investir 1000 a 1% ao mês por 12 meses')">Simular
investimento</button>
          <button onclick="fillExample('Como funciona cartão de crédito?')">Cartão de
crédito</button>
        </div>
      </div>

      <form id="chat-form" class="chat-form">
        <input type="text" id="message" placeholder="Digite sua mensagem..."
autocomplete="off" required>
        <button type="submit">Enviar</button>
        <button type="button" class="secondary" onclick="resetChat()">Reiniciar</button>
      </form>
    </main>
  </div>

  <script>
    const chatForm = document.getElementById('chat-form');
    const messageInput = document.getElementById('message');
    const chatBox = document.getElementById('chat-box');

    chatForm.addEventListener('submit', async (e) => {
      e.preventDefault();

      const message = messageInput.value.trim();
```

```
      if (!message) return;

      addMessage(message, 'user');
      messageInput.value = '';

      const response = await fetch('/chat', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ message })
      });

      const data = await response.json();
      addMessage(data.reply, 'bot');
    });

    function addMessage(text, sender) {
      const wrapper = document.createElement('div');
      wrapper.className = `message ${sender}`;

      const bubble = document.createElement('div');
      bubble.className = 'bubble';
      bubble.innerHTML = text.replace(/\n/g, '<br>');

      wrapper.appendChild(bubble);
      chatBox.appendChild(wrapper);
      chatBox.scrollTop = chatBox.scrollHeight;
    }

    function fillExample(text) {
      messageInput.value = text;
      messageInput.focus();
    }

    async function resetChat() {
      const response = await fetch('/reset', {
        method: 'POST'
      });

      const data = await response.json();
      chatBox.innerHTML = `
        <div class="message bot">
          <div class="bubble">
            Conversa reiniciada com sucesso.<br><br>
            Olá. Eu sou o <strong>FinAssist AI</strong>.<br><br>
            Posso te ajudar com:
            <ul>
              <li>explicação de produtos financeiros</li>
              <li>simulação de empréstimo</li>
              <li>simulação de investimento</li>
              <li>dúvidas gerais sobre relacionamento financeiro</li>
            </ul>
          </div>
        </div>
      `;
    }
  </script>
</body>
</html>
```

static/style.css

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: Arial, sans-serif;
}

body {
  background: #f4f7fb;
  color: #1f2937;
}

.container {
  max-width: 900px;
  margin: 0 auto;
  padding: 24px;
}

.header {
  text-align: center;
  margin-bottom: 24px;
}

.header h1 {
  color: #0f172a;
  margin-bottom: 8px;
}

.header p {
  color: #475569;
}

.chat-wrapper {
  background: #ffffff;
  border-radius: 16px;
  box-shadow: 0 4px 18px rgba(0, 0, 0, 0.08);
  padding: 20px;
}

.chat-box {
  height: 420px;
  overflow-y: auto;
  border: 1px solid #e5e7eb;
  border-radius: 12px;
  padding: 16px;
  background: #f8fafc;
  margin-bottom: 16px;
}

.message {
  display: flex;
  margin-bottom: 12px;
}

.message.user {
  justify-content: flex-end;
}

.message.bot {
  justify-content: flex-start;
}

.bubble {
```

```
  max-width: 75%;
  padding: 12px 14px;
  border-radius: 14px;
  line-height: 1.5;
  font-size: 14px;
}

.message.user .bubble {
  background: #2563eb;
  color: white;
  border-bottom-right-radius: 4px;
}

.message.bot .bubble {
  background: #e2e8f0;
  color: #0f172a;
  border-bottom-left-radius: 4px;
}

.examples {
  margin-bottom: 16px;
}

.examples p {
  margin-bottom: 10px;
  color: #334155;
}

.example-list {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
}

.example-list button {
  border: none;
  background: #dbeafe;
  color: #1d4ed8;
  padding: 10px 12px;
  border-radius: 999px;
  cursor: pointer;
  transition: 0.2s;
}

.example-list button:hover {
  background: #bfdbfe;
}

.chat-form {
  display: flex;
  gap: 10px;
}

.chat-form input {
  flex: 1;
  padding: 14px;
  border: 1px solid #cbd5e1;
  border-radius: 10px;
  outline: none;
}

.chat-form button {
  border: none;
  background: #0f172a;
  color: white;
  padding: 14px 18px;
```

```
  border-radius: 10px;
  cursor: pointer;
  transition: 0.2s;
}

.chat-form button:hover {
  opacity: 0.92;
}

.chat-form .secondary {
  background: #64748b;
}

ul {
  padding-left: 18px;
  margin-top: 8px;
}

@media (max-width: 768px) {
  .chat-form {
    flex-direction: column;
  }

  .bubble {
    max-width: 90%;
  }
}
```

12. Melhorias Futuras

- Integração com modelo de IA generativa (GPT, Claude, etc.)
- Persistência em banco de dados (PostgreSQL, MongoDB)
- Autenticação de usuários
- Dashboard administrativo
- Integração com APIs reais do setor financeiro
- Personalização por perfil de usuário
- Análise de sentimento
- Suporte a múltiplos idiomas
- Deploy em produção (AWS, Azure, Heroku)

13. Conclusão

O **FinAssist AI** representa uma solução completa e funcional para o desafio proposto, integrando inteligência artificial, Python, dados e experiência do usuário em um contexto financeiro.

O projeto demonstra capacidade técnica, visão de produto, preocupação com segurança e clareza na comunicação, características essenciais para soluções digitais voltadas ao relacionamento com o cliente.

A arquitetura modular permite evolução contínua, tornando o projeto escalável e preparado para integração com tecnologias mais avançadas no futuro.